

Git - version control system.

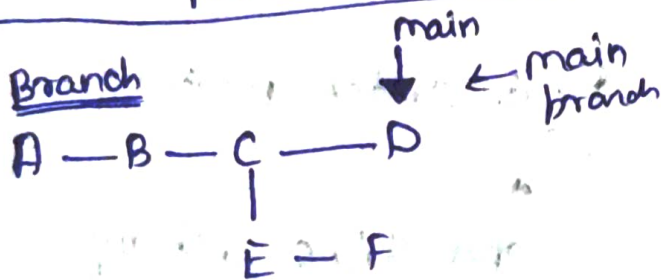
↳ tracks code history
(records the changes)

```
git add
git commit -m "login page"
```

commit changes →

↳ can compare prev versions and find bug

Control: we can control over previous version



1. Folder

Project

↳ index.html

↳ style.css

2. Tracking this project

```
git init
```

* it creates hidden folder that stores the info it needs to manage the repo and its history
project that git tracking
(not only folder)

3. modify index.html
<h1> Welcome </h1>

```
git status
```

modified: index.html (in red)

4. Staging area: which changes to save

```
git add index.html
```

5. commit modified code

```
git commit -m "Welcome"
```

6. inspect history

```
git log
```

online

```
git log --online
```

o/p

commit Head → master
(current)

Styled well

commit

created welcome page.

local repo to git repo

```
git push
```

github repo changes to local pc

```
git pull
```

git clone ← clone github proj to local pc

edit files

git add ← select changes

git commit ← save a version

git push → upload commit to github.

Git configuration details

git config --global user.name
git config --global user.email

O/P:
name
mail id

To check project folder connected to github

git remote -v

O/P:
if nothing -> no
if yes -> shows
origin -> proj link

Branch:

a movable pointer to a commit

commit A -> commit B -> commit C
↑
feature (branch)
master (branch)

git branch

O/P:
* master

git branch feature
git branch

O/P:
feature
* master

both points commit C

git switch feature
git branch

O/P:
* feature
master

* current working branch feature

git switch -c feature
create branch and switch

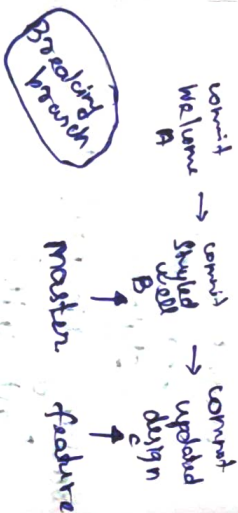
modify file [Now branch on feature]

1. git status
-> modified: style.css
2. git add style.css
3. git commit -m "updated data"
3. Look history

git log --oneline --graph --all

git switch master

↑
open style.css, changes
will disappear.



Merge branch:

git switch master
git merge feature

A -- B -- C

↑
master
feature

Delete unused branch

git branch -d model

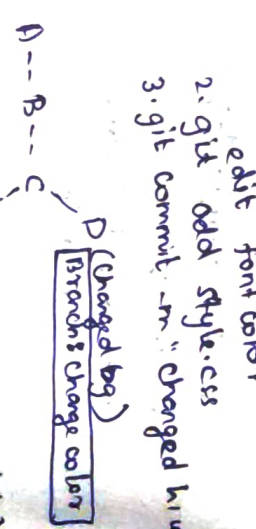
* can't able to delete current branch

-d VS -D -> force delete even unmerged

branch with already merged commits
protection of delete work
avoidance of delete work

1. git switch -c change color
edit style.css
2. git add style.css
3. git commit -m "changed by"

1. git switch master
edit font color
2. git add style.css
3. git commit -m "changed by"



git merge change-color

↳ vim editor

esc

:wq
exit editor

git status: master

git log shows the branch

merge conflict:

while changing two branches on same line and merging back.

git switch -c conflict-test

edit by color to purple
git add style.css
git commit -m "set by purple"

git switch master

edit by color to pink
git add style.css
git commit -m "set by pink"

git merge conflict-test

O/P:
CONFLICT: ...

Automatic merge failed...

In style.css

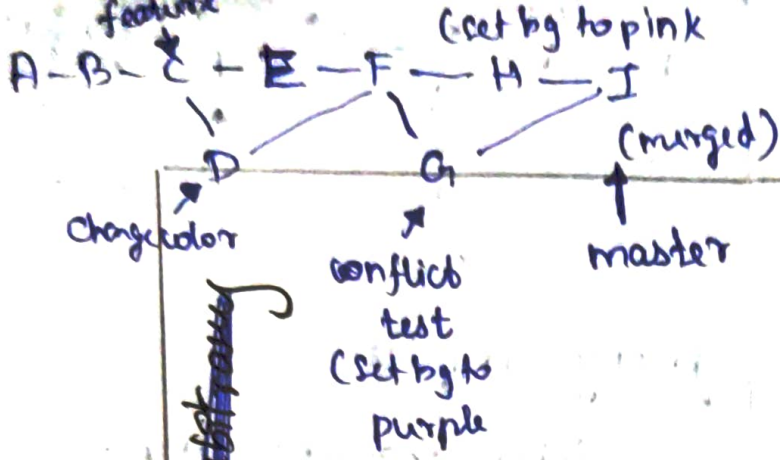
<<< HEAD
this line (pink by color)

===
purple by color line

conflict-test

replace code with needed
git add style.css, git commit -m "conflict"





③ git add style.css
It is now in staging area

④ git status
modified: style.css

⑤ git diff
Shows nothing

Reason:
Show changes b/w working directory and staging area

⑥ git diff --staged
(before staged what change)

O/P
- prev font size
+ current changes

⑦ git diff HEAD
all the changes staged and unstaged

Note: diff is before commit

diff
① edit font size - prev font size
+ current changes

git diff

②
Staged -> git add -f

* Show what changed but haven't staged yet

--- = old file
+++ = new file

git status
↳ which file changed

git diff
↳ which line changed

2b branch
A-B-C-D
if i need to add branch at B,

then, git switch -c -name-
id of that branch
git switch -c ui a4fda45

git ignore

↳ don't track these files or folders

add all the folders inside

git ignore

and run

git status

git ignore

← file

• secret/
• env/

↳ folder names to hide

git restore

1. modify index.html

2. no need that change

git restore index.html

* it will restore the part where change done ~~to~~ [Before committing changed file]

* If already staged and need to restore the prev committed file then

git restore --staged index.html

After the commit, need the previous committed file

git revert

Steps:

A - B

git revert id of A

eg:

1. git log - online
from this we can get

2. git revert cf91a53
if goes vim editor
i) esc ii) :wq iii) Enter

git reset ⇒ moves branch to another commit

A - B - C - D

↑
vi
(HEAD)

↑
master

HEAD ⇒ currently working

i) modify index.html

ii) git add index.html

git commit -m "Added def"

git reset HEAD~1

* move Head to vi, the changes will remain

check with git status

→ modified: index.html

3 mode

mixed reset

1. git reset --HEAD

commit X
staging X
working file ✓

2. git reset --soft HEAD~1

commit X
staging ✓
working file ✓

3. git reset --hard HEAD~1

commit X
staging X
working file X

git stash → save uncommitted changes

when to use?

If need to move to other branch still want the current code.

git stash list → show the list of stashed branch

git stash pop → restore changes and remove stash

[Show what you change in stash]

If need the prev committed code

git restore style.css

i) modify font face

git stash

ii) open style.css shows old committed font

* add color to P and commit changes.
iii) now need that font to this file then,

git stash apply

now the stash will apply here

Conflicts will be similar to merge conflicts

Can have multiple stash

stash@{0}

stash@{1}

stash@{2}

stash@{3}

can't apply everything add at once - apply 1 after other

git stash apply stash@{0}

git stash clear → will clean all the stash

git stash drop stash@{1}

↳ delete specific stash

git stash -u

↳ untracked files also included

git stash -a

↳ untracked files and ignored files also included

Rebase:

A → B → C → D
↑
vi master

git switch vi

git rebase master

A -- B -- C -- D

↑ code will be moved vi, master

if Branch/REBASE 1/3)

again

git add index.html

git rebase --continue

A → B → C → D
↑
E + master
vi git rebase master
A → B → E → C' → D'
↑
master vi

rebase:

A → added git vs git hub

→ B → added features → added commands
vi C E

git rebase master

A → B → C' → E'
↑
master vi

git fetch

↳ downloads latest changes and commits from remote repository
↳ not apply to current branch

GitHub repository

1. create repo in github

2. git remote add origin-link [taking link to add code]

3. git push -u origin vi

PC
vi branch
2actf9c
2actf9c

`git branch -vv`

↳ shows branch in github
change code in github

`git fetch`

download the changes in
github

`git diff ui origin/ui`

↳ shows what changes

to return \$

`press q`

3 `git pull`

↳ apply the changes from
GH to your local PC code

`git log --oneline --graph -all -5`

↳ last 5 history

cherry-pick

↳ take one commit from
another branch and
apply to current branch

`git cherry-pick <commit-id-of
branch-commit>`

conflicts like

`<<< HEAD`

`>>> 44d7c0b`

After fix

`git add index.html`

`git cherry-pick --continue`

Don't want those cherry-pick

`git cherry-pick --abort`

git branch -a → Shows all branch name
local + remote

Git Remote Branches:

git remote -v
* Shows online repo

git status -sb
* Shows status short branch

git branch -vv
* Shows local + remote branches
with the commit message
remote-tracking branches

git branch -r
* Shows remote branch
push new branch to github.

**git push -u origin
cherry-pick-practice**

-u ⇒ set upstream

* new branch in local to github
origin / cherry-pick-practice

git remote show origin

fetch url:

push url:

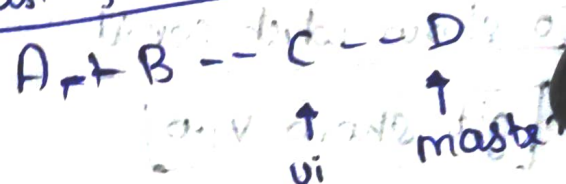
Head branch

Remote branch

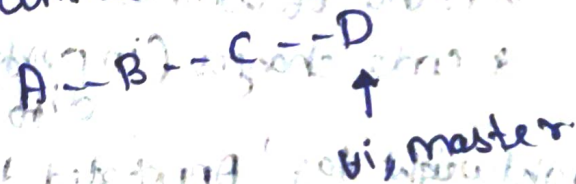
local branch for git pull

git push

Fast-forward:



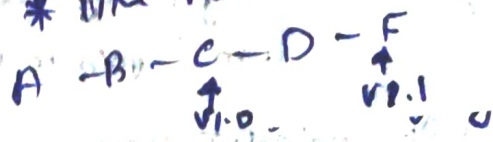
Move a branch pointer to
other without any merge/
commit



Tag (Fixed label)

* A name/label attached to
specific commit.

* like versions



git tag v1.0

* on a current branch tag

o/p: v1.0

* if success shows nothing

To verify

git tag

o/p: v1.0

To show all tags
To show which commit

git show v1.0

o/p:

* commit id, origin

* Author

* Date

* Name & commit

* code changes (in bold diff)

Lightweight tag

Annotated tag

label as v1.0

contains extra info: tag name, tag msg, tagger, date.

git tag v1.0

git tag -a v1.0 -m "version 1.1"

To delete a tag:

git tag -d v1.0

can able to move a tag

git tag -f p-tag 85d4e11

-f => force

o/p: updated tag 'p-tag' (was ...)

To get commit hash:

git rev-list -n 1 p-tag

o/p:

85d4e1177670678202... tag

cat file -> what type & object (compact tag) tag points to?

git cat-file -t v1.0

o/p: commit (lightweight tag)

git cat-file -t v1.1

o/p: tag (annotated tag)

git cat-file -p v1.1

object hash

type commit

tag v1.0

tagger vashu

commit name.

Tags on Github:

git ls-remote --tags origin

* tags on github

To push tags:

git push origin --tags

* all local tags are pushed to github

git push origin v1.0

* push specific tag

reflog:

* keeps a local record of where branch / HEAD pointers have moved - including all actions

* useful when lose commit and want to recover

actions

* useful when lose commit and want to recover

git reflog

↳ shows all

HEAD

HEAD {1}

git show HEAD{4}

HEAD@{4}

↳ not 4th commit

* position of HEAD 4

reflog entry 4 ago

git reflog -n 5

↳ recent 5 reflog.

`git reflog ui`

* shows history of specific branch

`git reflog ui -n 5`

* specific branch recent 5 history of HEAD

Note:

* old reflog entries may expired if no used longer time

Reflog = temporary safety record where references used to point

Detached HEAD:

HEAD from branch to commit

`git switch --detach 85dea27`

from : GitLearning (ui)

to : GitLearning (85dea27-)

* Instead of pointing branch directly pointing commit

* on a detached head when committed, and switch back to branch

A → B → C ← ui

↓
85dea27

↓
(new commit)
124f20

* Move back to ui

* None is pointing to 85dea27
using reflog we can access 124f20

* Return head back by switch

`git switch ui`

If on detached head need

commit to save:

Create branch

`git switch -c New-branch`

Multiple collaborators

1. search repo
2. git clone https://.....
3. cd GitLearning
4. git remote -v
* shows origin link
eg: Vedharshi/GitLearning.git
5. From second acct
make change & commit
then

`git show --stat`

`git show --online`

6. Pushing changes to owner
account

`git push origin ui`

from main acct do
changes and try to push
it shows err
cause conflicts.

1. git pull
2. solve merge conflict
3. And push it again

Pull Requests:

1. From second account
clone... do changes in
code
2. Push the changes.
3. Open the second account
see the pull requests
4. click compare & pull req
5. create pull request

→ In first account

↳ pull requests → click.

↳ files changes
(can see code)

(if it ok)

↳ submit review

⊙ Approve

↳ then merge.